

InterLens Package & API Specification

This document details the Python package organization, module structures, class hierarchies, API endpoints, and configuration schemas for `interlens`.

1. Directory & Module Structure

```
interlens/  
├── pyproject.toml      # Build config, dependencies, entry points  
├── README.md          # Project overview & quickstart  
├── docs/              # Architecture & specs  
│   ├── ARCHITECTURE.md  
│   ├── PACKAGE_SPEC.md  
│   ├── UI_SPEC.md  
│   └── ROADMAP.md  
├── plan/  
│   └── transformer-debugger-plan.md  
└── interlens/         # Main Python package  
    ├── __init__.py    # Package exports & version  
    ├── cli.py         # Command-line interface (`interlens launch`)  
    ├── config.py      # Global configuration & environment settings  
    ├── schema.py      # Pydantic schemas / DTOs for API  
    ├── adapters/      # Layer 2: Model Adapters  
    │   ├── __init__.py  
    │   ├── base.py    # BaseModelAdapter abstract interface  
    │   ├── inplace.py # InPlaceModelAdapter for pre-loaded GPU models  
    │   ├── custom.py  # CustomModelAdapter & AutoPyTorchHook for novel models  
    │   ├── gpt2.py    # GPT-2 family adapter  
    │   ├── llama.py   # Llama-3 family adapter  
    │   └── qwen.py    # Qwen2.5 family adapter  
    ├── analysis/      # Layer 3: Core Interpretability Engines  
    │   ├── __init__.py  
    │   ├── logit_lens.py # Logit Lens computations  
    │   ├── residual_stream.py # Norms, variances, PCA trajectories  
    │   ├── attention_heads.py # Attention matrices & arc data  
    │   ├── neurons.py   # Neuron ranking & lighting strips  
    │   ├── causal_patching.py # Clean/Corrupt activation patching  
    │   ├── causal_tracing.py # Multi-layer/position causal sweeps  
    │   ├── sae_explorer.py # SAE feature decomposition  
    │   ├── attribution.py # Input token attribution  
    │   └── comparison.py # Side-by-side model diffing  
    ├── server/        # Layer 4 Backend: FastAPI Server  
    │   ├── __init__.py  
    │   ├── app.py     # FastAPI instance setup & middleware  
    │   ├── session.py # LRU Cache Manager for activation runs  
    │   └── routes/  
    │       ├── model.py # Model Loading & listing routes  
    │       ├── analysis.py # Interpretability data routes  
    │       └── patching.py # Intervention & experiment routes  
    └── ui/            # Layer 4 Frontend: Static Web UI Assets  
        ├── index.html  
        ├── assets/  
        └── dist/      # Pre-built dashboard frontend
```

2. Core Python Classes & Interfaces

2.1 BaseModelAdapter (`interplens/adapters/base.py`)

```

from abc import ABC, abstractmethod
from typing import List, Dict, Any, Optional, Tuple
import torch
from transformer_lens import HookedTransformer

class BaseModelAdapter(ABC):
    """Abstract base class standardizing model interactions for InterPlens."""

    def __init__(self, model_name: str, device: str = "cpu"):
        self.model_name = model_name
        self.device = device
        self.model: Optional[HookedTransformer] = None

    @abstractmethod
    def load(self) -> None:
        """Loads model into memory/device."""
        pass

    @abstractmethod
    def tokenize(self, text: str) -> List[str]:
        """Converts string prompt into list of formatted token labels."""
        pass

    @abstractmethod
    def run_with_cache(self, prompt: str) -> Tuple[torch.Tensor, Any]:
        """Runs forward pass and returns (Logits, ActivationCache)."""
        pass

    @abstractmethod
    def get_resid_post_hook_name(self, layer: int) -> str:
        """Returns standard hook name for residual stream post-layer."""
        pass

    @abstractmethod
    def get_attn_pattern_hook_name(self, layer: int) -> str:
        """Returns standard hook name for attention pattern matrix."""
        pass

```

2.2 Session Store (`interplens/server/session.py`)

```

class SessionStore:
    """LRU Cache store maintaining computed ActivationCaches in memory."""

    def create_session(self, model_name: str, prompt: str) -> str:
        """Runs forward pass, caches activations, returns unique session_id."""
        pass

    def get_session(self, session_id: str) -> Dict[str, Any]:
        """Retrieves cached logits, tokens, and ActivationCache."""
        pass

```

2.3 InPlaceModelAdapter & Python Researcher API (`interplens/adapters/inplace.py`)

When a researcher already has a model instance loaded in GPU memory (e.g. `HookedTransformer` or `nn.Module`), `interplens` attaches directly to the existing object without re-downloading or re-allocating VRAM weights:

```
import interplens as il
from transformer_lens import HookedTransformer

# 1. Researcher's existing model on CUDA
model = HookedTransformer.from_pretrained("gpt2-small", device="cuda")

# 2. Launch InterPlens UI attaching to existing model instance (zero-copy VRAM)
il.launch(model=model, port=8501)

# 3. Or use programmatically in Jupyter / Python script:
session = il.inspect(model=model, prompt="The Eiffel Tower is in")
logits = session.logit_lens()
attention = session.attention_patterns(layer=3)
```

2.4 Custom Model Adapters & PyTorch Auto-Hooking (`interplens/adapters/custom.py`)

For researchers building novel architecture variants or custom PyTorch `nn.Module` models:

Option A: Subclassing `BaseModelAdapter`

Researchers can define custom hook mappings in under 10 lines of code:

```
import interplens as il

class MyNovelModelAdapter(il.BaseModelAdapter):
    def __init__(self, custom_model, tokenizer):
        super().__init__(model_name="ExperimentalModel-v1")
        self.model = custom_model
        self.tokenizer = tokenizer

    def tokenize(self, text: str) -> List[str]:
        return self.tokenizer.tokenize(text)

    def run_with_cache(self, prompt: str):
        # Executes custom forward pass and captures activation dict
        return self.model.forward_with_cache(prompt)
```

Option B: Automated PyTorch `nn.Module` Auto-Hooker (`PyTorchAutoHooker`)

For raw PyTorch models that do not use `TransformerLens` natively, `interplens` uses PyTorch's native `register_forward_hook()` to automatically intercept hidden states across `nn.ModuleList` blocks:

```
# Auto-hooks any PyTorch nn.Module architecture
il.launch(model=my_custom_nn_module, tokenizer=my_tokenizer, auto_hook=True)
```

3. FastAPI REST Endpoints

Method	Endpoint	Description
GET	/api/health	Server status and active PyTorch device (CPU/CUDA/MPS).
GET	/api/models	List supported & currently loaded model adapters.
POST	/api/models/load	Load a model (e.g. <code>gpt2</code> , <code>Qwen/Qwen2.5-0.5B</code>).
POST	/api/run	Execute forward pass on prompt & generate <code>session_id</code> .
GET	/api/analysis/logit-lens	Fetch Logit Lens predictions across all layers.
GET	/api/analysis/residual-stream	Fetch residual norms & PCA trajectories.
GET	/api/analysis/attention	Fetch attention matrices for specified layer/head.
GET	/api/analysis/neurons	Fetch top firing neurons for a selected token.
POST	/api/experiments/patch	Run clean vs. corrupted activation patching.
POST	/api/experiments/causal-trace	Run automated layer/position causal sweep.
GET	/api/analysis/sae	Fetch SAE feature activations for target layer/token.

4. Dependencies & CLI Specification

Dependencies (`pyproject.toml`)

- `torch >= 2.0.0`
- `transformer-lens >= 2.0.0`
- `fastapi >= 0.100.0`
- `uvicorn >= 0.20.0`
- `pydantic >= 2.0.0`
- `einops >= 0.6.0`
- `jaxtyping >= 0.2.0`
- `scikit-learn >= 1.2.0` (for PCA)

CLI Command (`interplens launch`)

```
# Basic Launch  
interplens launch  
  
# Launch with pre-loaded model on specific port  
interplens launch --model gpt2 --port 8501 --device cuda
```